CivicRecords AI

Open-source, locally-hosted AI that helps American cities respond to open records requests.

CivicRecords AI runs entirely on a single machine inside your city's network — no cloud subscriptions, no vendor lock-in, no resident data leaving the building. It ingests your city's documents, makes them searchable with AI-powered natural language queries, detects potential exemptions, and manages the full request lifecycle from intake to response.

Why This Exists

Every city in America processes open records requests (FOIA, CORA, and state equivalents). Staff manually search file shares, email archives, and databases — then review every document for exemptions before release. It's slow, error-prone, and a growing burden as request volumes increase.

No open-source tool exists for the **responder side** of open records at the municipal level. CivicRecords AI fills that gap.

Key Features

- **AI-Powered Search** — Natural language hybrid search (semantic + keyword) across all ingested city documents with source attribution, normalized relevance scores, and optional AI-generated summaries
- **Document Ingestion** — Automatic parsing of PDF, DOCX, XLSX, CSV, email, HTML, and text files. Scanned documents processed via multimodal AI (Gemma 4) with Tesseract OCR fallback
- **Exemption Detection** — Tier 1 PII detection (SSN, credit card with Luhn validation, phone, email, bank accounts, state-specific driver's licenses) plus per-state statutory keyword matching. Optional LLM secondary review. All flags require human confirmation
- **Request Management** — Full lifecycle tracking with 10 statuses: received, clarification needed, assigned, searching, in review, ready for release, drafted, approved, fulfilled, closed. Timeline, messaging, fee tracking, and response letter generation
- **Guided Onboarding** — 3-phase wizard helps cities configure their profile, identify data systems across 12 municipal domains, and surface coverage gaps

- **Municipal Systems Catalog** — Curated knowledge base of 25+ municipal software vendors across 12 functional domains (finance, public safety, permitting, HR, etc.) with discovery hints and connector templates
- **Universal Connector Framework** — Standardized protocol (authenticate/discover/fetch/health_check) for connecting to city data sources. Ships with four implemented connector types: `file_system` (local/mounted directories), `manual_drop` (watched drop folders), `rest_api` (generic REST API — API key / Bearer / OAuth2 client-credentials / Basic auth; JSON/XML/CSV; page/offset/cursor pagination), and `odbc` (SQL databases via pyodbc, row-as-document with SQL-injection guards). IMAP email, SMB/NFS, and SharePoint connectors on roadmap
- **Scheduled Sync & Idempotent Ingestion** — Per-source cron scheduling (5-field expressions via croniter, evaluated in UTC with local-time disclosure, rolling 7-day min-interval validation, 5-minute floor) with `schedule_enabled` pause toggle. Idempotent pipeline: binary sources dedup by content hash, structured REST/ODBC sources dedup by stable source-path with canonical JSON serialization. Concurrent-update races prevented via `SELECT FOR UPDATE` + partial UNIQUE indexes; content updates atomically replace chunks and embeddings in the same transaction
- **Sync Failure Tracking & Circuit Breaker** — Per-record failure tracking with two-layer retry (task-level exponential backoff + record-level per-tick retry with N=100/T=90s cap). Automatic circuit breaker after 5 consecutive full-run failures (`sync_paused`) with admin-feedback unpause grace period. `health_status` (healthy/degraded/circuit_open) computed live from failure counts. Admin UI: colored health badge, failed records panel with bulk retry/dismiss, Sync Now button with real-time polling progress
- **Operational Analytics** — Real-time metrics: average response time, deadline compliance rate, overdue requests, status breakdown
- **Notification Service** — Template-based notification system with SMTP email delivery via Celery beat (60s interval). Configure SMTP_HOST, SMTP_PORT, SMTP_USERNAME, SMTP_PASSWORD in .env to enable. Notification dispatch into status transitions pending
- **Compliance by Design** — Hash-chained audit logs, human-in-the-loop enforcement, AI content labeling, data sovereignty verification. Designed for Colorado CAIA and 50-state regulatory compliance. CJIS compliance gate for public safety connectors
- **Civic Design System** — Professional UI built with shadcn/ui, civic blue design tokens, sidebar navigation. WCAG 2.2 AA targeted (44px touch targets, skip navigation, icon+color status badges — accessibility audit pending)
- **Federation-Ready** — REST API with service accounts enables future cross-jurisdiction record discovery between CivicRecords AI instances

Quick Start

Requirements

- **Docker Desktop** (Windows 10/11, macOS 13+) or **Docker Engine** (Linux)
- **8+ CPU cores**, **32 GB RAM**, **50 GB free disk space**
- No internet connection required after initial setup

Install

Windows: ``` powershell git clone https://github.com/scottconverse/civicrecords-ai.git cd civicrecords-ai .\install.ps1 ```

macOS / Linux: ```bash git clone https://github.com/scottconverse/civicrecords-ai.git cd civicrecords-ai bash install.sh ```

First Use

1. Open **http://localhost:8080** in your browser 2. Sign in with the admin credentials you configured in .env 3. Go to **Sources** → **Add Source** → enter a directory path to your documents 4. Click **Ingest Now** — documents are parsed, chunked, and indexed automatically 5. Go to **Search** — type a natural language query and get cited results

Architecture

```
┌──────────────┐ ┌──────────────┐ ┌──────────────┐ │ Browser │──▶│ nginx │──▶│ FastAPI │ │ (React UI) │ │ (frontend) │ │ (API) │ └──────────────┘ └──────────────┘ └──────┬───────┘ └──────────────┘ │ ┌──────────────────────────┘ │ │ │ ┌──────▼──────┐ ┌──────▼──────┐ ┌──────▼──────┐ │ PostgreSQL │ │ Redis │ │ Ollama │ │ + pgvector │ │ (queue) │ │ (LLM) │ └─────────────┘ └─────────────┘ └─────────────┘ │ ┌──────▼──────┐ │ Celery │ │ (worker) │ └─────────────┘
```

7 Docker services: PostgreSQL 17 + pgvector, Redis 7.2, Ollama, FastAPI, Celery worker, Celery beat, nginx frontend.

Tech stack: Python 3.12, FastAPI, SQLAlchemy 2.0, React 18, shadcn/ui, Tailwind CSS, Alembic, Celery, pgvector, nomic-embed-text, Gemma 4 (recommended).

Configuration

All configuration is via environment variables in `.env`:

Variable	Description	Default
`DATABASE_URL`	PostgreSQL connection string	`postgresql+asyncpg://civicrecords:civicrecords@postgres:5432/civicrecords`
`JWT_SECRET`	Secret key for JWT tokens	(must be set)
`FIRST_ADMIN_EMAIL`	Initial admin account email	`admin@example.gov`
`FIRST_ADMIN_PASSWORD`	Initial admin account password	(must be set — min 12 chars; installer generates automatically; `.env.example` placeholder and common defaults rejected at startup)
`OLLAMA_BASE_URL`	Ollama API endpoint	`http://ollama:11434`
`REDIS_URL`	Redis connection string	`redis://redis:6379/0`
`AUDIT_RETENTION_DAYS`	Audit log retention period	`1095` (3 years)
`CONNECTOR_HOST_ALLOWLIST`	Comma-separated hostnames/IPs exempt from SSRF block (on-prem use)	(empty — RFC1918, loopback, and cloud IMDS ranges blocked by default)

Supported Platforms

- Windows 10/11 (Docker Desktop)
- macOS 13+ (Docker Desktop)
- Ubuntu 22.04+ / Debian 12+ (Docker Engine)

All platforms use identical Docker containers — the application runs in Linux containers regardless of host OS.

Data Sovereignty

CivicRecords AI is designed for environments where resident data must never leave the network:

- Runs entirely on local hardware — no cloud dependencies
- No telemetry, analytics, or crash reporting

- All LLM inference runs locally via Ollama
- `Verification script confirms no outbound connections:` bash scripts/verify-sovereignty.sh

License

Apache License 2.0 — see LICENSE.

All dependencies use permissive (MIT, Apache 2.0, BSD) or weak-copyleft (LGPL, MPL) licenses. No AGPL, SSPL, or BSL dependencies.

Documentation

Complete System Manual (unified staff + IT admin guide with architecture diagrams):

- Download PDF | Download Word | View Online

Individual References:

- Staff User Manual — For city clerks and records officers (non-technical)
- IT Administrator Manual — Installation, configuration, security, backup, monitoring
- Canonical Spec — Unified Design Specification (single source of truth)
- System Architecture Diagram — Interactive component and data flow diagrams
- Phase Decomposition — Project phases and build sequence

Contributing

See CONTRIBUTING.md for development setup, coding standards, and how to submit changes.

Testing Prerequisites

`Integration tests require Docker Compose with PostgreSQL running, and the` civicrecords `Postgres user needs DROP/CREATE DATABASE privileges (the default docker compose config does this automatically). Each test drops and recreates` civicrecords_test `to guarantee a clean schema.`

`bash docker compose up -d postgres redis docker compose run --rm api python -m pytest tests -q`

If a prior test run died mid-fixture, civicrecords_test may already exist and the suite will fail with DuplicateDatabaseError. Drop it manually and re-run:

`bash docker compose exec postgres dropdb -U civicrecords civicrecords_test`

User Roles

| Role | Permissions | Phase | |------|-------------|-------| | **Admin** | Full access: user management, system config, rule management, audit logs, onboarding | Built | | **Staff** | Search, create requests, attach documents, scan for exemptions, review flags, manage fees | Built | | **Reviewer** | Everything Staff can do + approve/reject responses and exemption flags | Built | | **Read-Only** | View search results and request status only | Built | | **Liaison** | Department-scoped read access to requests and search; Users/Audit Log/Onboarding nav hidden | Built | | **Public** | Submit requests, track own requests, search published records | v1.1 |

Service accounts with hashed API keys enable instance-to-instance federation access.

Status

v1.1.0 — Phase 2 release with department access controls, 50-state exemption rules, and compliance templates.

New in v1.1.0:

- Department-level access controls — staff scoped to own department, admins see all
- Department CRUD API with audit logging
- 50-state + DC exemption rule coverage (180 rules across 51 jurisdictions)
- 5 compliance template documents (AI Use Disclosure, Response Letter Disclosure, CAIA Impact Assessment, AI Governance Policy, Data Residency Attestation)
- Template render endpoint with city profile variable substitution
- Exemption auditability dashboard with acceptance/rejection rates and CSV/JSON export (time-period filtering not yet implemented)
- Model registry CRUD endpoints (spec 6.7 compliance metadata)

Carried from v1.0.x:

- 13 staff workbench pages + Login with shadcn/ui design system
- 29 database tables, ~30 API endpoints
- 556 backend + 7 frontend automated tests passing (GitHub Actions CI-verified)
- Guided onboarding, systems catalog, connector framework
- Request timeline, messaging, fee tracking, response letter generation
- Operational analytics and notification service
- AMD GPU/NPU hardware auto-detection (ROCm on Linux, DirectML on Windows)
- Login rate limiting, audit log archival, admin-only user creation
- Tested on Windows 11 (Docker Desktop) and Ubuntu 22.04 (Docker Engine)

Post-v1.1.0 security hardening (on master, not yet released):

- **T2A** — Role self-escalation via PATCH /users/me closed (UserSelfUpdate schema); all 24 department-scoped handlers enforce require_department_scope; 404/403 status-code info-leak unified via require_department_or_404; Pattern D list-endpoint fail-open fixed on 4 routes; parameterized enforcement test covers 25 routes
- **T2B** — connection_config redacted from GET /datasources/ for all non-admin users (DataSourceAdminRead returned only on admin write endpoints). **ENG-001 runtime exposure: closed. At-rest storage (plaintext JSONB in DB) remains open — tracked as Tier 6.**
- **T2C** — FIRST_ADMIN_PASSWORD startup validator (rejects .env.example placeholder, empty, <12 chars, and common defaults); SSRF host validator blocks loopback, IMDS, and RFC1918 ranges on REST and ODBC connector URLs at schema-validation time
- **T3A** — Create-user form now POSTs to /api/admin/users (was /api/auth/register, which silently downgraded submitted roles to STAFF)
- **CI** — 556 backend + 7 frontend tests; bootstrap-failure smoke test confirms stack rejects placeholder admin password

> **ENG-001 standing caveat:** At-rest encryption for data_sources.connection_config is not yet implemented. The JSONB column is visible to any database superuser,

pg_dump` output, or restored backup. ENG-001 will not be fully closed until Tier 6 at-rest encryption lands.

Roadmap (per canonical spec):

Phase	Focus	Key Deliverables	Status
Phase 0	Design foundation	shadcn/ui, design tokens, sidebar shell, component library	In progress
Phase 1	Staff workbench redesign	Redesign all staff pages with new design system, WCAG 2.2 AA	In progress
Phase 2	New backend features	Fees, notifications (SMTP), response letters, context manager, analytics, liaison role, department scoping, audit retention	Partially built
Phase 3	Public portal	Public homepage, search, guided request wizard, request tracker, help pages	Planned (v1.1)
Phase 4	Transparency layer	Open records library, reporting dashboards, public archive, federation	Planned (v2.0)

Note: Version numbers (semver) track release history. Phase numbers track design completeness per the canonical spec. They are separate systems. Current build (v1.1.0) includes backend work from Phases 0-2 but has not completed the full scope of any phase. See canonical spec for complete requirements and reconciliation for current gap analysis.